

Техническое задание

CRM-система для производства и продаж изделий из спанбонда

1. Общая идея проекта

Необходимо разработать веб-приложение CRM/ERP-lite для компании, которая производит и продает изделия из спанбонда: простыни, дастарханы, рулоны, упаковочные изделия и другие товары.

Главный приоритет системы — удобный ввод данных с телефона для сотрудников в цеху, мастеров смены, менеджеров и закупщика. Руководитель и администратор должны получать детальную аналитику на компьютере через dashboard.

Система должна работать как адаптивное веб-приложение:

- на телефоне — быстрые формы, крупные кнопки, минимум полей, удобная работа одной рукой;
- на ПК — таблицы, фильтры, графики, аналитика, управление справочниками и отчетами.

2. Цели CRM-системы

Система должна решать следующие задачи:

1. Контроль заказов клиентов.
2. Учет производства по сменам.
3. Автоматический складской учет сырья и готовой продукции.
4. Учет отгрузок и продаж.
5. Учет оплат от клиентов.
6. Автоматический расчет задолженности клиентов.
7. Учет всех расходов компании.
8. Формирование P&L-отчетности: выручка, расходы, прибыль.
9. Контроль действий сотрудников.
10. Удобная работа с телефона без Excel и бумажных записей.

3. Роли пользователей

3.1. Super Admin

Полный доступ ко всей системе.

Права:

- управление пользователями;
- управление ролями;
- просмотр всех заказов;
- просмотр всех смен;
- утверждение отчетов смен;
- управление складом;
- просмотр финансов;
- просмотр P&L;
- удаление/редактирование критичных данных;
- настройка справочников.

3.2. Admin / Руководитель

Почти полный доступ, кроме технических настроек.

Права:

- заказы;
- производство;
- склад;
- продажи;
- оплаты;
- расходы;
- отчеты;
- аналитика;
- утверждение сменных отчетов.

3.3. Менеджер продаж

Права:

- создание клиентов;
- создание заказов;
- изменение статусов заказов;
- фиксация отгрузок;
- внесение оплат;
- просмотр задолженности своих клиентов;
- просмотр остатков готовой продукции.

Не имеет доступа к расходам, зарплатам, полной прибыли.

3.4. Мастер смены

Права:

- создание отчета смены;
- выбор работников смены;
- указание произведенного товара;

- указание потраченного сырья;
- указание брака и остатков;
- просмотр своих сменных отчетов.

Не может сам утверждать отчет смены, если это не разрешено настройками.

3.5. Складовщик

Права:

- просмотр склада;
- фиксация ручных корректировок склада;
- прием сырья;
- выдача сырья;
- подтверждение отгрузки готовой продукции.

3.6. Закупщик

Права:

- добавление расходов на закуп сырья;
- добавление закупленных материалов;
- пополнение склада сырья через расход.

3.7. Рабочий

Минимальный доступ.

Права:

- видеть назначенные задачи/смены;
- отмечать участие в смене;
- при необходимости отмечать выполненную работу.

4. Рекомендуемый стек технологий

4.1. Backend

Основной стек:

- Python 3.12+
- FastAPI
- PostgreSQL
- SQLAlchemy 2.0 Async
- Alembic
- Pydantic v2
- JWT Auth
- Passlib / bcrypt

- Uvicorn / Gunicorn
- Docker / Docker Compose

Почему этот стек подходит:

- надежный для бизнес-систем;
- PostgreSQL хорошо подходит для финансов, склада и отчетов;
- FastAPI быстро разрабатывать;
- удобно делать REST API;
- легко подключить аналитику, роли, фильтры и экспорт в Excel.

4.2. Frontend

Рекомендуемый стек:

- Next.js
- React
- TypeScript
- Tailwind CSS
- shadcn/ui
- Zustand или TanStack Query
- React Hook Form
- Zod
- Recharts для графиков
- TanStack Table для таблиц

Почему:

- хороший responsive UI;
- удобно делать mobile-first интерфейс;
- можно сделать PWA, чтобы приложение выглядело как мобильное приложение;
- удобно разворачивать как веб-сайт.

4.3. База данных

- PostgreSQL 15/16+
- Индексы по датам, статусам, client_id, order_id, product_id
- UUID для primary key
- Decimal/Numeric для денег и количества

4.4. Инфраструктура

Минимальный вариант:

- VPS сервер
- Docker Compose
- PostgreSQL container или managed PostgreSQL
- Caddy или Nginx
- SSL через Let's Encrypt
- GitHub Actions для деплоя

4.5. Хранение файлов

На первом этапе можно без файлов.

На втором этапе:

- MinIO или S3-compatible storage;
 - загрузка чеков, накладных, фото брака, фото отгрузки.
-

5. Основные функциональные блоки

Блок 1. Заказы клиентов

Назначение

Менеджер создает заказ клиента. Заказ проходит статусы от нового до отгруженного или отмененного.

Статусы заказа

- NEW — Новый
- IN_PRODUCTION — В производстве
- READY_TO_SHIP — Готов к отгрузке
- SHIPPED — Отгружен
- CANCELLED — Отменен

Поля заказа

- клиент;
- номер заказа;
- дата создания;
- дедлайн;
- статус;
- комментарий;
- общая сумма заказа;
- менеджер;
- позиции заказа.

Позиции заказа

Каждый заказ может содержать несколько товаров:

- простыни;
- дастарханы;
- рулоны;
- другая готовая продукция.

Для каждой позиции:

- товар;
 - количество;
 - цена за единицу;
 - сумма;
 - комментарий.
-

Блок 2. Отчеты смен

Назначение

Мастер смены с телефона заполняет отчет по производству.

Отчет должен быть максимально простым:

1. Выбрать смену: Смена 1 или Смена 2.
2. Выбрать дату.
3. Выбрать работников.
4. Указать, что произвели.
5. Указать, какое сырье потратили.
6. Указать брак.
7. Отправить на утверждение.

Статусы отчета смены

- DRAFT — Черновик
- SUBMITTED — Отправлен на проверку
- APPROVED — Утвержден
- REJECTED — Отклонен

Важная логика

Пока отчет смены не утвержден, склад не меняется.

После утверждения отчета:

- сырье автоматически списывается со склада;
 - готовая продукция автоматически добавляется на склад;
 - брак фиксируется отдельной складской операцией или в отчете.
-

Блок 3. Складской учет

Назначение

Система должна показывать актуальные остатки:

1. Готовой продукции.

2. Сырья и расходников.

Типы складских товаров

Готовая продукция:

- дастархан;
- простыня;
- спанбонд;
- другие изделия.

Сырье и расходники:

- рулоны спанбонда;
- тубусы;
- красители;
- упаковочная пленка;
- краска;
- другие материалы.

Главный принцип склада

Остаток не должен храниться только как одно число, которое постоянно редактируется вручную.

Правильная логика:

- есть таблица товаров/materials/products;
- есть таблица складских движений stock_movements;
- текущий остаток считается как сумма приходов минус сумма расходов.

Можно дополнительно хранить cached quantity в таблице stock_balances для скорости, но главным источником правды должен быть журнал складских операций.

Типы складских движений

- PURCHASE_IN — приход сырья от закупки;
- PRODUCTION_IN — приход готовой продукции после смены;
- PRODUCTION_OUT — списание сырья на производство;
- SALE_OUT — списание готовой продукции при продаже;
- ADJUSTMENT_IN — ручная корректировка в плюс;
- ADJUSTMENT_OUT — ручная корректировка в минус;
- DEFECT_OUT — списание брака;
- RETURN_IN — возврат.

Блок 4. Продажи и отгрузки

Назначение

Продажа фиксируется в момент фактической отгрузки товара со склада.

Отгрузка должна быть привязана к заказу клиента.

Логика

Когда заказ отгружается:

1. Менеджер или складовщик создает отгрузку.
2. Выбирает заказ.
3. Выбирает товары и количество.
4. Система проверяет остаток готовой продукции.
5. Если товара хватает — создается отгрузка.
6. Склад готовой продукции уменьшается.
7. Заказ получает статус SHIPPED или частично отгружен, если будет добавлен такой статус.

Важно

В P&L выручку можно считать двумя способами:

1. По отгрузкам — начисленная выручка.
2. По оплатам — кассовая выручка.

В настройках dashboard нужно дать переключатель:

- “Считать выручку по отгрузкам”
 - “Считать выручку по оплатам”
-

Блок 5. Оплаты от клиентов

Назначение

Фиксация входящих платежей от клиентов.

Способы оплаты

- CASH — Наличные
- BANK_TRANSFER — Безналичный расчет
- CARD — Карта
- OTHER — Другое

Поля оплаты

- клиент;
- заказ, необязательно;
- дата платежа;
- сумма;
- способ оплаты;
- комментарий;
- кто внес оплату в систему.

Логика

Оплата может быть:

- привязана к конкретному заказу;
 - просто привязана к клиенту как общий платеж.
-

Блок 6. Дебиторская задолженность

Назначение

Автоматический расчет долгов клиентов.

Формула

Долг клиента = сумма всех отгрузок клиента минус сумма всех оплат клиента.

Если долг больше 0 — клиент должен компании.

Пример:

- клиенту отгрузили товара на 1 000 000 ₺;
- клиент оплатил 700 000 ₺;
- долг = 300 000 ₺.

Экран задолженности

Таблица:

- клиент;
- телефон;
- менеджер;
- сумма отгрузок;
- сумма оплат;
- текущий долг;
- дата последней оплаты;
- дата последней отгрузки;
- кнопка “Добавить оплату”;
- кнопка “Открыть историю”.

Сортировка по умолчанию:

- от самого большого долга к меньшему.
-

Блок 7. Расходы

Назначение

Учет всех затрат компании.

Категории расходов

Категории должны быть кастомным справочником, чтобы администратор мог добавлять новые.

Базовые категории:

1. Закуп сырья и расходников:
2. спанбонд;
3. тубусы;
4. пленка;
5. краска;
6. упаковка.
7. Операционные расходы:
8. аренда цеха;
9. коммунальные услуги;
10. логистика;
11. доставка.
12. Фонд оплаты труда:
13. зарплаты рабочим;
14. бонусы менеджерам;
15. премии;
16. авансы.
17. Амортизация и ремонт оборудования:
18. станки;
19. принтеры;
20. ремонт;
21. обслуживание.
22. Прочие расходы.

Поля расхода

- дата;
- категория;
- сумма;

- способ оплаты;
- комментарий;
- поставщик, необязательно;
- created_by;
- является ли расход закупкой сырья;
- список закупленных материалов, если это закуп сырья.

Важная связь закупки сырья со складом

Если расход относится к закупке сырья или расходников, пользователь должен иметь возможность добавить позиции закупки:

- какой материал купили;
- количество;
- единица измерения;
- цена за единицу;
- общая сумма.

После сохранения и подтверждения такого расхода:

- в расходах появляется сумма;
- на складе сырья автоматически создается приход;
- создается stock_movement типа PURCHASE_IN.

Пример:

Расход:

- категория: Закуп сырья
- сумма: 500 000 ₴
- поставщик: ТОО “Поставщик”
- способ оплаты: Безнал

Позиции:

- рулон спанбонда белый — 20 рулонов
- тубусы — 100 штук
- пленка упаковочная — 50 кг

После подтверждения:

- расходы +500 000 ₴;
- склад сырья пополнен;
- в журнале склада появились движения PURCHASE_IN.

Блок 8. Отчетность P&L

Назначение

Dashboard для руководителя.

Периоды

Пользователь должен выбирать:

- сегодня;
- неделя;
- месяц;
- квартал;
- год;
- произвольный период.

Показатели

1. Валовая выручка.
2. Полученные оплаты.
3. Сумма отгрузок.
4. Общие расходы.
5. Чистая прибыль.
6. Дебиторская задолженность.
7. Остатки готовой продукции.
8. Остатки сырья.
9. Производство за период.
10. Брак за период.

Формулы

Валовая выручка по отгрузкам:

```
gross_revenue = SUM(shipments.total_amount)
```

Валовая выручка по оплатам:

```
cash_revenue = SUM(payments.amount)
```

Общие расходы:

```
total_expenses = SUM(expenses.amount)
```

Чистая прибыль по отгрузкам:

```
net_profit = gross_revenue - total_expenses
```

Чистая прибыль по оплатам:

```
cash_profit = cash_revenue - total_expenses
```

Дебиторская задолженность:

```
accounts_receivable = SUM(shipments.total_amount) - SUM(payments.amount)
```

Графики

На dashboard должны быть:

1. Линейный график доходов и расходов по датам.
2. Круговая диаграмма расходов по категориям.
3. Столбчатый график производства по товарам.
4. Карточки KPI:
5. выручка;
6. расходы;
7. прибыль;
8. долги клиентов;
9. произведено товаров;
10. брак.

6. Архитектура базы данных

Важно: 8 функциональных блоков не равны 8 таблицам. Для нормальной CRM нужно больше таблиц, потому что должны быть связи, история, позиции заказов, движения склада, пользователи и справочники.

Ниже нормализованная структура PostgreSQL.

6.1. users

Пользователи системы.

Поля:

```
id UUID PK
full_name VARCHAR(255)
phone VARCHAR(50)
email VARCHAR(255) UNIQUE
password_hash TEXT
role VARCHAR(50)
is_active BOOLEAN DEFAULT true
```

```
created_at TIMESTAMP
updated_at TIMESTAMP
```

Роли:

```
SUPER_ADMIN
ADMIN
MANAGER
SHIFT_MASTER
WAREHOUSE
PURCHASER
WORKER
```

6.2. clients

Клиенты.

```
id UUID PK
name VARCHAR(255)
phone VARCHAR(50)
email VARCHAR(255)
company_name VARCHAR(255)
address TEXT
comment TEXT
manager_id UUID FK -> users.id
created_at TIMESTAMP
updated_at TIMESTAMP
```

Связи:

- один клиент может иметь много заказов;
- один клиент может иметь много оплат;
- один клиент может иметь много отгрузок.

6.3. products

Готовая продукция.

```
id UUID PK
name VARCHAR(255)
sku VARCHAR(100) UNIQUE
category VARCHAR(100)
unit VARCHAR(50)
default_price NUMERIC(14,2)
```

```
is_active BOOLEAN DEFAULT true
created_at TIMESTAMP
updated_at TIMESTAMP
```

Примеры:

- простыня 160x200;
- дастархан;
- спанбонд рулон;
- упаковка.

6.4. materials

Сырье и расходники.

```
id UUID PK
name VARCHAR(255)
sku VARCHAR(100) UNIQUE
category VARCHAR(100)
unit VARCHAR(50)
is_active BOOLEAN DEFAULT true
created_at TIMESTAMP
updated_at TIMESTAMP
```

Примеры:

- рулон спанбонда;
- тубус;
- пленка;
- краситель;
- краска;
- упаковка.

6.5. orders

Заказы клиентов.

```
id UUID PK
order_number VARCHAR(100) UNIQUE
client_id UUID FK -> clients.id
manager_id UUID FK -> users.id
status VARCHAR(50)
deadline DATE
comment TEXT
total_amount NUMERIC(14,2) DEFAULT 0
```

```
created_at TIMESTAMP
updated_at TIMESTAMP
```

Статусы:

```
NEW
IN_PRODUCTION
READY_TO_SHIP
SHIPPED
CANCELLED
```

6.6. order_items

Позиции заказа.

```
id UUID PK
order_id UUID FK -> orders.id ON DELETE CASCADE
product_id UUID FK -> products.id
quantity NUMERIC(14,3)
unit_price NUMERIC(14,2)
total_price NUMERIC(14,2)
comment TEXT
created_at TIMESTAMP
updated_at TIMESTAMP
```

Логика:

```
total_price = quantity * unit_price
orders.total_amount = SUM(order_items.total_price)
```

6.7. shift_reports

Отчеты смен.

```
id UUID PK
shift_date DATE
shift_type VARCHAR(50)
master_id UUID FK -> users.id
status VARCHAR(50)
comment TEXT
approved_by UUID FK -> users.id NULL
approved_at TIMESTAMP NULL
```



```
created_at TIMESTAMP
updated_at TIMESTAMP
```

shift_type:

```
SHIFT_1
SHIFT_2
```

status:

```
DRAFT
SUBMITTED
APPROVED
REJECTED
```

6.8. shift_report_workers

Работники, участвовавшие в смене.

```
id UUID PK
shift_report_id UUID FK -> shift_reports.id ON DELETE CASCADE
worker_id UUID FK -> users.id
hours_worked NUMERIC(6,2)
comment TEXT
created_at TIMESTAMP
```

6.9. shift_report_outputs

Что произвели за смену.

```
id UUID PK
shift_report_id UUID FK -> shift_reports.id ON DELETE CASCADE
product_id UUID FK -> products.id
quantity NUMERIC(14,3)
defect_quantity NUMERIC(14,3) DEFAULT 0
comment TEXT
created_at TIMESTAMP
```

Пример:

- простыня — 500 шт;
- брак — 12 шт.

6.10. shift_report_materials

Какое сырье потратили за смену.

```
id UUID PK
shift_report_id UUID FK -> shift_reports.id ON DELETE CASCADE
material_id UUID FK -> materials.id
quantity_used NUMERIC(14,3)
waste_quantity NUMERIC(14,3) DEFAULT 0
comment TEXT
created_at TIMESTAMP
```

Пример:

- спанбонд — 120 кг;
- пленка — 10 кг;
- тубусы — 30 шт.

6.11. warehouses

Склады.

На первом этапе можно создать один склад, но архитектура должна поддерживать несколько.

```
id UUID PK
name VARCHAR(255)
type VARCHAR(50)
is_active BOOLEAN DEFAULT true
created_at TIMESTAMP
updated_at TIMESTAMP
```

type:

```
RAW_MATERIALS
FINISHED_GOODS
MIXED
```

6.12. stock_movements

Главная таблица складских движений.

```
id UUID PK
warehouse_id UUID FK -> warehouses.id
item_type VARCHAR(50)
product_id UUID FK -> products.id NULL
material_id UUID FK -> materials.id NULL
movement_type VARCHAR(50)
quantity NUMERIC(14,3)
unit VARCHAR(50)
unit_cost NUMERIC(14,2) NULL
total_cost NUMERIC(14,2) NULL
source_type VARCHAR(50)
source_id UUID NULL
comment TEXT
created_by UUID FK -> users.id
created_at TIMESTAMP
```

item_type:

```
PRODUCT
MATERIAL
```

movement_type:

```
PURCHASE_IN
PRODUCTION_IN
PRODUCTION_OUT
SALE_OUT
ADJUSTMENT_IN
ADJUSTMENT_OUT
DEFECT_OUT
RETURN_IN
```

source_type:

```
EXPENSE
SHIFT_REPORT
SHIPMENT
MANUAL_ADJUSTMENT
RETURN
```

Очень важное правило:

- если item_type = PRODUCT, product_id обязателен, material_id должен быть NULL;
- если item_type = MATERIAL, material_id обязателен, product_id должен быть NULL.

6.13. stock_balances

Кэш текущих остатков.

```
id UUID PK
warehouse_id UUID FK -> warehouses.id
item_type VARCHAR(50)
product_id UUID FK -> products.id NULL
material_id UUID FK -> materials.id NULL
quantity NUMERIC(14,3) DEFAULT 0
updated_at TIMESTAMP
```

Эта таблица обновляется автоматически при создании stock_movements.

Можно пересчитать остатки из stock_movements, если возникнет ошибка.

6.14. shipments

Отгрузки клиентам.

```
id UUID PK
shipment_number VARCHAR(100) UNIQUE
order_id UUID FK -> orders.id
client_id UUID FK -> clients.id
warehouse_id UUID FK -> warehouses.id
shipment_date DATE
status VARCHAR(50)
total_amount NUMERIC(14,2)
comment TEXT
created_by UUID FK -> users.id
created_at TIMESTAMP
updated_at TIMESTAMP
```

status:

```
DRAFT
CONFIRMED
CANCELLED
```

6.15. shipment_items

Позиции отгрузки.

```
id UUID PK
shipment_id UUID FK -> shipments.id ON DELETE CASCADE
product_id UUID FK -> products.id
quantity NUMERIC(14,3)
unit_price NUMERIC(14,2)
total_price NUMERIC(14,2)
created_at TIMESTAMP
```

После подтверждения shipment:

- создается stock_movement SALE_OUT;
- уменьшается stock_balances;
- заказ может перейти в SHIPPED.

6.16. payments

Оплаты от клиентов.

```
id UUID PK
client_id UUID FK -> clients.id
order_id UUID FK -> orders.id NULL
payment_date DATE
amount NUMERIC(14,2)
payment_method VARCHAR(50)
comment TEXT
created_by UUID FK -> users.id
created_at TIMESTAMP
updated_at TIMESTAMP
```

payment_method:

```
CASH
BANK_TRANSFER
CARD
OTHER
```

6.17. expense_categories

Категории расходов.

```
id UUID PK
name VARCHAR(255)
type VARCHAR(50)
```

```
is_inventory_related BOOLEAN DEFAULT false
is_active BOOLEAN DEFAULT true
created_at TIMESTAMP
updated_at TIMESTAMP
```

type:

```
RAW_MATERIAL_PURCHASE
OPERATING
PAYROLL
EQUIPMENT
OTHER
```

6.18. expenses

Расходы.

```
id UUID PK
expense_date DATE
category_id UUID FK -> expense_categories.id
amount NUMERIC(14,2)
payment_method VARCHAR(50)
supplier_name VARCHAR(255) NULL
comment TEXT
status VARCHAR(50)
created_by UUID FK -> users.id
approved_by UUID FK -> users.id NULL
approved_at TIMESTAMP NULL
created_at TIMESTAMP
updated_at TIMESTAMP
```

status:

```
DRAFT
SUBMITTED
APPROVED
REJECTED
CANCELLED
```

6.19. expense_items

Позиции закупки сырья внутри расхода.

Эта таблица нужна только для расходов, которые связаны с закупкой сырья или расходников.

```
id UUID PK
expense_id UUID FK -> expenses.id ON DELETE CASCADE
material_id UUID FK -> materials.id
warehouse_id UUID FK -> warehouses.id
quantity NUMERIC(14,3)
unit VARCHAR(50)
unit_price NUMERIC(14,2)
total_price NUMERIC(14,2)
created_at TIMESTAMP
```

Логика:

```
total_price = quantity * unit_price
```

При утверждении расхода:

- создается stock_movement PURCHASE_IN по каждой позиции;
- stock_balances увеличивается;
- expense.status становится APPROVED.

6.20. audit_logs

История действий пользователей.

```
id UUID PK
user_id UUID FK -> users.id
action VARCHAR(100)
entity_type VARCHAR(100)
entity_id UUID
old_value JSONB NULL
new_value JSONB NULL
created_at TIMESTAMP
```

Примеры action:

```
CREATE_ORDER
UPDATE_ORDER_STATUS
APPROVE_SHIFT_REPORT
CREATE_EXPENSE
APPROVE_EXPENSE
CREATE_PAYMENT
```

CONFIRM_SHIPMENT
MANUAL_STOCK_ADJUSTMENT

7. Основные связи между таблицами

```
users 1 -> many clients
clients 1 -> many orders
orders 1 -> many order_items
products 1 -> many order_items

users 1 -> many shift_reports
shift_reports 1 -> many shift_report_workers
shift_reports 1 -> many shift_report_outputs
shift_reports 1 -> many shift_report_materials

products 1 -> many stock_movements
materials 1 -> many stock_movements
warehouses 1 -> many stock_movements

orders 1 -> many shipments
shipments 1 -> many shipment_items
shipment_items many -> 1 products

clients 1 -> many payments
orders 1 -> many payments

expense_categories 1 -> many expenses
expenses 1 -> many expense_items
expense_items many -> 1 materials

expenses -> stock_movements через source_type = EXPENSE
shift_reports -> stock_movements через source_type = SHIFT_REPORT
shipments -> stock_movements через source_type = SHIPMENT
```

8. Ключевая бизнес-логика

8.1. Создание заказа

1. Менеджер создает клиента или выбирает существующего.
2. Создает заказ.
3. Добавляет товары.
4. Указывает количество, цену, дедлайн.
5. Система считает сумму.
6. Заказ получает статус NEW.

8.2. Перевод заказа в производство

1. Менеджер или админ меняет статус на IN_PRODUCTION.
2. Заказ отображается в списке производственных заказов.
3. Мастер смены видит, какие заказы нужно выполнить.

На первом этапе можно не делать сложное планирование производства. Достаточно видеть список заказов в производстве.

8.3. Создание отчета смены

1. Мастер смены открывает мобильную форму.
 2. Выбирает дату и смену.
 3. Выбирает работников.
 4. Добавляет произведенные товары.
 5. Добавляет потраченное сырье.
 6. Указывает брак.
 7. Нажимает "Отправить на проверку".
 8. Статус отчета становится SUBMITTED.
-

8.4. Утверждение отчета смены

Админ открывает отчет и проверяет данные.

Если все правильно:

1. Нажимает "Утвердить".
2. Система проверяет, хватает ли сырья на складе.
3. Если сырья хватает:
 4. создает stock_movements PRODUCTION_OUT для сырья;
 5. создает stock_movements PRODUCTION_IN для готовой продукции;
 6. создает DEFECT_OUT для брака, если нужно;
 7. обновляет stock_balances;
 8. меняет статус отчета на APPROVED.
9. Если сырья не хватает:
10. система показывает ошибку;
11. отчет не утверждается.

Если данные неправильные:

- админ нажимает "Отклонить";

- указывает комментарий;
 - мастер смены исправляет отчет.
-

8.5. Закуп сырья через расходы

1. Закупщик или руководитель создает расход.
 2. Выбирает категорию "Закуп сырья".
 3. Указывает сумму.
 4. Добавляет позиции закупки:
 5. материал;
 6. количество;
 7. цена;
 8. склад.
 9. Отправляет на утверждение.
 10. Админ утверждает расход.
 11. Система:
 12. фиксирует расход;
 13. создает stock_movements PURCHASE_IN;
 14. увеличивает остатки сырья.
-

8.6. Отгрузка товара

1. Менеджер или складовщик открывает заказ.
 2. Нажимает "Создать отгрузку".
 3. Указывает товары и количество.
 4. Система проверяет остаток готовой продукции.
 5. Если товара хватает:
 6. отгрузка подтверждается;
 7. создается stock_movement SALE_OUT;
 8. остаток готовой продукции уменьшается;
 9. заказ получает статус SHIPPED.
 10. Если товара не хватает:
 11. система показывает ошибку;
 12. отгрузка не создается.
-

8.7. Оплата от клиента

1. Менеджер открывает клиента или заказ.
 2. Нажимает “Добавить оплату”.
 3. Указывает:
 4. дату;
 5. сумму;
 6. способ оплаты;
 7. комментарий.
 8. Система сохраняет платеж.
 9. Долг клиента автоматически пересчитывается.
-

8.8. Расчет долга клиента

```
client_debt = SUM(shipments.total_amount WHERE client_id = X AND status =  
CONFIRMED)  
-  
SUM(payments.amount WHERE client_id = X)
```

Если `client_debt > 0`, клиент должен компании.

9. Backend API

9.1. Auth

```
POST /api/auth/login  
POST /api/auth/refresh  
POST /api/auth/logout  
GET /api/auth/me
```

9.2. Users

```
GET /api/users  
POST /api/users  
GET /api/users/{id}  
PATCH /api/users/{id}  
DELETE /api/users/{id}
```

9.3. Clients

```
GET    /api/clients
POST   /api/clients
GET    /api/clients/{id}
PATCH /api/clients/{id}
DELETE /api/clients/{id}
GET    /api/clients/{id}/balance
GET    /api/clients/{id}/history
```

9.4. Products

```
GET    /api/products
POST   /api/products
GET    /api/products/{id}
PATCH /api/products/{id}
DELETE /api/products/{id}
```

9.5. Materials

```
GET    /api/materials
POST   /api/materials
GET    /api/materials/{id}
PATCH /api/materials/{id}
DELETE /api/materials/{id}
```

9.6. Orders

```
GET    /api/orders
POST   /api/orders
GET    /api/orders/{id}
PATCH /api/orders/{id}
DELETE /api/orders/{id}
PATCH /api/orders/{id}/status
```

Фильтры:

```
status
client_id
manager_id
```

```
date_from
date_to
deadline_from
deadline_to
search
```

9.7. Shift Reports

```
GET    /api/shift-reports
POST   /api/shift-reports
GET    /api/shift-reports/{id}
PATCH /api/shift-reports/{id}
DELETE /api/shift-reports/{id}
POST   /api/shift-reports/{id}/submit
POST   /api/shift-reports/{id}/approve
POST   /api/shift-reports/{id}/reject
```

9.8. Warehouses / Stock

```
GET    /api/warehouses
POST   /api/warehouses

GET    /api/stock/balances
GET    /api/stock/movements
POST   /api/stock/adjustments
GET    /api/stock/products
GET    /api/stock/materials
```

Фильтры:

```
warehouse_id
item_type
product_id
material_id
movement_type
date_from
date_to
```

9.9. Shipments

```
GET    /api/shipments
POST   /api/shipments
GET    /api/shipments/{id}
PATCH /api/shipments/{id}
POST   /api/shipments/{id}/confirm
POST   /api/shipments/{id}/cancel
```

9.10. Payments

```
GET    /api/payments
POST   /api/payments
GET    /api/payments/{id}
PATCH /api/payments/{id}
DELETE /api/payments/{id}
```

9.11. Expenses

```
GET    /api/expense-categories
POST   /api/expense-categories
PATCH /api/expense-categories/{id}

GET    /api/expenses
POST   /api/expenses
GET    /api/expenses/{id}
PATCH /api/expenses/{id}
DELETE /api/expenses/{id}
POST   /api/expenses/{id}/submit
POST   /api/expenses/{id}/approve
POST   /api/expenses/{id}/reject
```

9.12. Reports

```
GET /api/reports/dashboard
GET /api/reports/pnl
GET /api/reports/debts
GET /api/reports/production
GET /api/reports/stock
```

```
GET /api/reports/expenses-by-category
GET /api/reports/revenue-expense-trend
```

Параметры:

```
date_from
date_to
revenue_mode=shipments|payments
```

10. UI/UX структура приложения

10.1. Основной layout

Desktop

На ПК интерфейс должен иметь:

- левое боковое меню;
- верхнюю панель с пользователем;
- основной dashboard;
- таблицы с фильтрами;
- графики;
- экспорт данных.

Меню:

1. Dashboard
2. Заказы
3. Клиенты
4. Производство
5. Склад
6. Отгрузки
7. Оплаты
8. Долги
9. Расходы
10. Отчеты
11. Справочники
12. Пользователи
13. Настройки

Mobile

На телефоне:

- нижнее меню или компактное меню;
- крупные кнопки;
- формы в формате step-by-step;

- минимум таблиц;
- карточки вместо таблиц;
- быстрые действия.

Нижнее меню для мобильного:

1. Главная
 2. Заказы
 3. Смена
 4. Склад
 5. Еще
-

11. UI/UX по страницам

11.1. Dashboard

Desktop

Показывать:

- KPI-карточки:
- выручка;
- оплаты;
- расходы;
- прибыль;
- долги;
- заказы в производстве;
- готово к отгрузке;
- брак.
- график доходы/расходы;
- круговая диаграмма расходов;
- таблица последних заказов;
- таблица крупных долгов;
- остатки критичного сырья.

Mobile

Показывать:

- 4 главные карточки:
- Сегодня выручка;
- Сегодня расходы;
- Прибыль;
- Долги.
- быстрые кнопки:
- Добавить заказ;
- Отчет смены;
- Добавить расход;
- Добавить оплату.

- компактный график за неделю.
-

11.2. Заказы

Desktop

Таблица:

- номер;
- клиент;
- статус;
- сумма;
- дедлайн;
- менеджер;
- дата создания;
- действия.

Фильтры:

- статус;
- дата;
- клиент;
- менеджер;
- поиск.

Mobile

Карточки заказов:

- номер заказа;
 - клиент;
 - статус цветом;
 - сумма;
 - дедлайн;
 - кнопки:
 - открыть;
 - изменить статус;
 - отгрузить;
 - добавить оплату.
-

11.3. Форма заказа

Поля:

- клиент;
- дедлайн;
- товары;
- количество;

- цена;
- комментарий.

Mobile-first требования:

- добавление товара через большую кнопку “+ Добавить товар”;
 - автоподсчет суммы;
 - сохранение черновика;
 - минимум обязательных полей.
-

11.4. Отчет смены

Это одна из самых важных мобильных страниц.

Форма должна быть пошаговой.

Шаг 1. Смена

- дата;
- смена 1 / смена 2;
- мастер смены автоматически текущий пользователь.

Шаг 2. Работники

- список работников с чекбоксами;
- часы работы;
- быстрые кнопки:
- 8 часов;
- 12 часов.

Шаг 3. Произведено

Карточки:

- товар;
- количество;
- брак;
- комментарий.

Кнопка:

- “+ Добавить товар”.

Шаг 4. Потрачено сырья

Карточки:

- материал;
- количество;
- отходы;

- комментарий.

Кнопка:

- “+ Добавить сырье”.

Шаг 5. Проверка

Показать сводку:

- кто работал;
- что произвели;
- что потратили;
- брак;
- комментарий.

Кнопки:

- Сохранить черновик;
 - Отправить на проверку.
-

11.5. Утверждение смены

Для админа на ПК:

- список отчетов в статусе SUBMITTED;
 - детальный просмотр;
 - произведено;
 - потрачено;
 - брак;
 - предупреждение, если сырья недостаточно;
 - кнопки:
 - Утвердить;
 - Отклонить.
-

11.6. Склад

Desktop

Табы:

1. Готовая продукция.
2. Сырье.
3. Движения склада.
4. Корректировки.

Таблица остатков:

- товар/материал;

- склад;
- остаток;
- единица;
- последнее движение.

Фильтры:

- тип;
- склад;
- категория;
- поиск.

Mobile

Карточки остатков:

- название;
- остаток;
- единица;
- цветовой индикатор:
- зеленый — нормально;
- желтый — мало;
- красный — критично.

11.7. Отгрузки

Форма отгрузки:

- выбрать заказ;
- выбрать склад;
- товары из заказа подтянуть автоматически;
- указать фактически отгруженное количество;
- система показывает доступный остаток;
- кнопка “Подтвердить отгрузку”.

После подтверждения:

- склад уменьшается;
- создается продажа;
- заказ становится SHIPPED, если все отгружено.

11.8. Оплаты

Форма оплаты:

- клиент;
- заказ, необязательно;
- дата;

- сумма;
- способ оплаты;
- комментарий.

Mobile:

- крупное поле суммы;
 - быстрый выбор способа оплаты;
 - кнопка “Сохранить оплату”.
-

11.9. Долги

Desktop:

Таблица клиентов:

- клиент;
- менеджер;
- сумма отгрузок;
- сумма оплат;
- долг;
- последняя оплата;
- последняя отгрузка;
- действия.

Mobile:

Карточки:

- клиент;
 - долг крупным числом;
 - телефон;
 - кнопка “Позвонить”;
 - кнопка “Добавить оплату”.
-

11.10. Расходы

Форма быстрого расхода:

- дата;
- категория;
- сумма;
- способ оплаты;
- комментарий.

Если категория “Закуп сырья”:

Появляется блок “Позиции закупки”:

- материал;
- количество;
- цена;
- склад.

Кнопки:

- Сохранить черновик;
- Отправить;
- Утвердить, если роль позволяет.

11.11. Отчеты P&L

Desktop:

- выбор периода;
- переключатель выручки:
- по отгрузкам;
- по оплатам.
- KPI;
- графики;
- таблицы.

Mobile:

- выбор периода;
- карточки:
- выручка;
- расходы;
- прибыль;
- долги.
- один простой график;
- расходы по категориям списком.

12. Backend: важные технические требования

12.1. Деньги

Все денежные значения хранить как:

NUMERIC(14,2)

Не использовать float.

12.2. Количество

Количество хранить как:

```
NUMERIC(14,3)
```

Потому что могут быть:

- штуки;
- кг;
- метры;
- рулоны;
- литры.

12.3. Транзакции

Критичные операции должны выполняться в одной database transaction:

1. Утверждение отчета смены.
2. Утверждение закупки сырья.
3. Подтверждение отгрузки.
4. Ручная корректировка склада.

Пример:

Если при утверждении смены сырье списалось, но готовая продукция не добавилась — это ошибка. Поэтому вся операция должна быть атомарной.

12.4. Проверка отрицательных остатков

Система не должна позволять уйти в минус по складу, кроме случаев, когда админ специально разрешил это в настройках.

По умолчанию:

```
Запретить отрицательные остатки.
```

12.5. Soft Delete

Для важных сущностей лучше использовать soft delete:

```
deleted_at TIMESTAMP NULL
```

Особенно для:

- заказов;

- клиентов;
- расходов;
- оплат;
- отгрузок.

На первом этапе можно использовать `is_active`, но лучше сразу заложить `deleted_at`.

12.6. Audit Log

Все важные действия должны логироваться:

- кто создал заказ;
 - кто изменил статус;
 - кто утвердил смену;
 - кто внес расход;
 - кто утвердил расход;
 - кто подтвердил отгрузку;
 - кто внес оплату;
 - кто сделал корректировку склада.
-

13. Frontend: технические требования

13.1. Общие требования

- TypeScript обязателен.
- Все формы через React Hook Form.
- Валидация через Zod.
- Данные с backend через TanStack Query.
- UI-компоненты через shadcn/ui.
- Стили через Tailwind CSS.
- Таблицы через TanStack Table.
- Графики через Recharts.

13.2. Responsive

Минимальные breakpoint:

```
mobile: 360px+  
tablet: 768px+  
desktop: 1024px+  
large desktop: 1440px+
```

На mobile:

- карточки вместо таблиц;
- кнопки минимум 44px высотой;
- нижняя навигация;

- формы в один столбец;
- sticky submit button снизу.

На desktop:

- sidebar;
- таблицы;
- фильтры;
- графики;
- bulk actions.

13.3. PWA

Желательно сделать PWA:

- добавить manifest;
- иконку приложения;
- возможность “Добавить на главный экран”;
- базовое кэширование интерфейса.

Offline-режим можно оставить на второй этап.

14. Рекомендуемая структура backend-проекта

```
backend/  
  app/  
    main.py  
    core/  
      config.py  
      security.py  
      database.py  
      permissions.py  
    models/  
      user.py  
      client.py  
      product.py  
      material.py  
      order.py  
      shift_report.py  
      warehouse.py  
      stock.py  
      shipment.py  
      payment.py  
      expense.py  
      audit_log.py  
    schemas/  
      user.py
```

```
client.py
product.py
material.py
order.py
shift_report.py
warehouse.py
stock.py
shipment.py
payment.py
expense.py
report.py
api/
  deps.py
  routes/
    auth.py
    users.py
    clients.py
    products.py
    materials.py
    orders.py
    shift_reports.py
    warehouses.py
    stock.py
    shipments.py
    payments.py
    expenses.py
    reports.py
  services/
    auth_service.py
    order_service.py
    shift_report_service.py
    stock_service.py
    shipment_service.py
    payment_service.py
    expense_service.py
    report_service.py
    audit_service.py
  repositories/
    base.py
  migrations/
  tests/
Dockerfile
docker-compose.yml
```

15. Рекомендуемая структура frontend-проекта

```
frontend/  
  app/  
    login/  
    dashboard/  
    orders/  
    clients/  
    production/  
    stock/  
    shipments/  
    payments/  
    debts/  
    expenses/  
    reports/  
    settings/  
  components/  
    layout/  
    ui/  
    forms/  
    tables/  
    charts/  
    mobile/  
  features/  
    auth/  
    orders/  
    clients/  
    production/  
    stock/  
    expenses/  
    reports/  
  lib/  
    api.ts  
    auth.ts  
    utils.ts  
    validations/  
  stores/  
  types/  
  public/
```

16. MVP — первая версия проекта

В первую версию обязательно включить:

16.1. Auth и роли

- login;
- JWT;
- роли пользователей;
- защита API;
- защита страниц frontend.

16.2. Справочники

- пользователи;
- клиенты;
- товары;
- сырье;
- категории расходов;
- склады.

16.3. Заказы

- создание заказа;
- позиции заказа;
- статусы;
- список заказов;
- фильтры.

16.4. Сменные отчеты

- создание отчета;
- добавление работников;
- добавление произведенной продукции;
- добавление потраченного сырья;
- отправка на проверку;
- утверждение админом;
- автоматическое движение склада.

16.5. Склад

- остатки;
- движения;
- ручные корректировки;
- автоматические списания и приходы.

16.6. Расходы

- добавление расхода;
- категории;
- закуп сырья;
- утверждение закупки;
- автоматическое пополнение склада.

16.7. Отгрузки

- создание отгрузки;
- проверка остатков;
- списание готовой продукции;
- привязка к заказу.

16.8. Оплаты и долги

- добавление оплаты;
- расчет долга клиента;
- экран дебиторки.

16.9. Dashboard

- выручка;
 - расходы;
 - прибыль;
 - долги;
 - график доходов/расходов;
 - расходы по категориям.
-

17. План реализации проекта

Этап 1. Проектирование

Сделать:

1. ERD-схему базы данных.
2. Описание ролей.
3. Описание бизнес-процессов.
4. Макеты основных экранов.
5. Список API endpoints.

Результат этапа:

- утвержденная структура БД;
 - утвержденные статусы;
 - утвержденные роли;
 - понятный score MVP.
-

Этап 2. Backend foundation

Сделать:

1. FastAPI проект.

2. PostgreSQL подключение.
3. SQLAlchemy async.
4. Alembic migrations.
5. Auth JWT.
6. Users/Roles.
7. Permissions.
8. Базовая структура services/repositories.

Результат:

- backend запускается;
 - login работает;
 - роли работают;
 - база мигрируется.
-

Этап 3. Справочники

Сделать CRUD:

1. Клиенты.
2. Товары.
3. Сырье.
4. Склады.
5. Категории расходов.
6. Пользователи.

Результат:

- можно заполнить базовые данные для работы CRM.
-

Этап 4. Заказы

Сделать:

1. Создание заказа.
2. Позиции заказа.
3. Пересчет суммы.
4. Статусы.
5. Фильтры.
6. Страница заказа.

Результат:

- менеджер может вести заказы клиентов.
-

Этап 5. Склад

Сделать:

1. stock_movements;
2. stock_balances;
3. сервис движения склада;
4. проверку отрицательных остатков;
5. ручные корректировки;
6. историю движения.

Результат:

- склад работает как журнал операций.
-

Этап 6. Производство / сменные отчеты

Сделать:

1. Мобильную форму отчета смены.
2. Работников смены.
3. Произведенную продукцию.
4. Потраченное сырье.
5. Брак.
6. Отправку на проверку.
7. Утверждение.
8. Автоматическое списание сырья и приход готовой продукции.

Результат:

- производство связано со складом.
-

Этап 7. Расходы и закуп сырья

Сделать:

1. Расходы.
2. Категории.
3. Позиции закупки сырья.
4. Утверждение расхода.
5. Автоматическое пополнение склада сырья.

Результат:

- закупка сырья отражается и в расходах, и на складе.
-

Этап 8. Отгрузки и продажи

Сделать:

1. Создание отгрузки по заказу.
2. Проверку остатков.
3. Списание готовой продукции.
4. Сумму отгрузки.
5. Изменение статуса заказа.

Результат:

- продажа фиксируется через отгрузку.
-

Этап 9. Оплаты и дебиторка

Сделать:

1. Внесение оплат.
2. Привязку к клиенту/заказу.
3. Расчет долга.
4. Экран долгов.
5. Историю клиента.

Результат:

- видно, кто сколько должен.
-

Этап 10. Dashboard и P&L

Сделать:

1. KPI.
2. Фильтр периода.
3. Выручка по отгрузкам.
4. Выручка по оплатам.
5. Расходы.
6. Прибыль.
7. Графики.
8. Расходы по категориям.
9. Долги.

Результат:

- руководитель видит финансовую картину бизнеса.
-

Этап 11. Тестирование

Обязательно протестировать:

1. Создание заказа.
 2. Утверждение смены.
 3. Списание сырья.
 4. Приход готовой продукции.
 5. Закуп сырья через расход.
 6. Пополнение склада.
 7. Отгрузку.
 8. Списание готовой продукции.
 9. Оплату.
 10. Расчет долга.
 11. P&L.
 12. Права доступа.
-

Этап 12. Деплой

Сделать:

1. Docker Compose.
 2. PostgreSQL.
 3. Backend container.
 4. Frontend container.
 5. Nginx/Caddy.
 6. SSL.
 7. Backup базы.
 8. Seed initial data.
-

18. Что можно оставить на вторую версию

1. Фото чеков и накладных.
 2. Фото брака.
 3. Печать накладных PDF.
 4. Экспорт в Excel.
 5. Telegram/WhatsApp уведомления.
 6. Offline mode.
 7. Штрихкоды/QR-коды.
 8. Себестоимость по партиям.
 9. Планирование производства.
 10. Зарплата рабочих по выработке.
 11. Интеграция с 1С.
 12. Интеграция с Kaspi/банком.
 13. Уведомления о низком остатке сырья.
-

19. Важные правила для разработчика

1. Все операции склада делать через stock_movements.
 2. Не менять остатки напрямую без движения.
 3. Деньги хранить только в NUMERIC, не float.
 4. Критичные операции делать через database transaction.
 5. Утверждение смены должно быть атомарным.
 6. Утверждение закупки должно быть атомарным.
 7. Отгрузка должна проверять остатки.
 8. Долги клиентов считать автоматически.
 9. Все важные действия логировать.
 10. Mobile UX должен быть приоритетом для форм ввода.
-

20. Критерии готовности MVP

Проект считается готовым, если:

1. Пользователь может войти в систему.
 2. Роли и доступы работают.
 3. Менеджер может создать клиента и заказ.
 4. Мастер может создать отчет смены с телефона.
 5. Админ может утвердить отчет смены.
 6. После утверждения смены сырье списывается, готовая продукция добавляется.
 7. Закупщик может внести расход на сырье.
 8. После утверждения закупки склад сырья пополняется.
 9. Менеджер может создать отгрузку по заказу.
 10. После отгрузки склад готовой продукции уменьшается.
 11. Менеджер может внести оплату клиента.
 12. Система показывает долг клиента.
 13. Dashboard показывает выручку, расходы и прибыль.
 14. Все основные страницы нормально работают на телефоне и ПК.
-

21. Рекомендуемый порядок разработки для Claude/разработчика

Сначала реализовать backend:

1. Models.
2. Migrations.
3. Schemas.
4. Auth.
5. CRUD.
6. Services.
7. Stock logic.
8. Reports.

Потом frontend:

1. Auth pages.
2. Layout.
3. Dashboard.
4. Orders.
5. Shift reports mobile form.
6. Stock.
7. Expenses.
8. Shipments.
9. Payments.
10. Debts.
11. Reports.

Главный технический центр проекта — это складская логика. Ее нужно сделать правильно с самого начала.

Самые важные сущности:

- stock_movements;
- stock_balances;
- shift_reports;
- expenses;
- shipments;
- payments.

Если эти блоки сделать правильно, CRM будет стабильно работать и масштабироваться.